

Name: Sreeja Reddy Yeluru

1. Q-table implementation and data structure choice

I used a python dictionary for my Q-Table with keys representing all possible states in the game and the values being lists of Q-values for each action that can be taken from that state. I made this choice because the state space is sparse, and we encounter states dynamically as the game plays out. Using a dictionary means we can add new states as we visit them. This is memory efficient vs. having a large fixed table. Additionally, this representation could be improved upon in several ways:

- A defaultdict to make initializing the table easier
- Compressing/hashing the states to use less memory
- Approximating the function mapping states to q-values (such as through a neural net) if the number of states gets too big

2. How Q-values are updated (with example)

I used the standard Q-learning update rule, where the value of a particular state-action pair is adjusted based on the immediate reward received and the estimated future rewards from the next state. The formula is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$


Where:

- s is the current state
- a is the action taken in state s
- r is the reward received after taking action a
- s' is the resulting next state

- α is the learning rate
- γ is the discount factor
- $\max_{a'} Q(s', a')$ is the best estimated value for the next state

Each time the agent takes an action, it receives a reward and updates the Q-value for that (state, action) pair.

Example 1 - Eating an Apple

Suppose the snake is at state s_1 , and it moves right to reach the apple. It receives a +10 reward.

- Previous $Q(s_1, \text{right}) = 2.0$
- $\max Q(s', a') = 3.5$
- $\alpha = 0.7$
- $\gamma = 0.95$
- $r = +10$

New $Q(s_1, \text{right}) = 2.0 + 0.7 \times [10 + 0.95 \times 3.5 - 2.0]$

New $Q(s_1, \text{right}) = 9.9275$

This updated Q-value reflects that taking the action “right” from s_1 is highly beneficial.

Example 2 - Hitting a Wall

Now suppose the snake is in state s_2 , and it moves left and hits the wall. It receives a -10 penalty.

- $Q(s_2, \text{left}) = 1.5$
- $\max Q(s', a') = 0.0$ (since it's a terminal state)
- $r = -10$

New $Q(s_2, \text{left}) = 1.5 + 0.7 \times [-10 + 0 - 1.5]$

New $Q(s_2, \text{left}) = -6.55$

Now, the Q-value reflects that taking action “left” from state s_2 is highly unfavorable.

3. Results and comments for Part 1 experiments(Observed noticeable differences in the snake behavior by different derived Q-tables)

In Part 1, I trained the snake agent multiple times using different values of epsilon and different numbers of training steps, which resulted in different derived Q-tables.

I observed clear and noticeable differences in the snake's behavior when evaluating each learned Q-table.

a) Q-table trained with epsilon = 0.2, num_steps = 500 (qtable_2025_2.csv)

Training output:

Return = -5.252, Apples = 14, Stops = 15, GoodSteps = 398, UniqueStatesVisited = 71

Evaluation output:

Return = 23.058, Apples = 23, Stops = 6, GoodSteps = 430, UniqueStatesVisited = 72

Observed behavior:

During evaluation, the snake was unable to find the apple by moving in a direct line towards the apple. Rather than finding the shortest route to the apple the snake often moved around the apple and would go up and/or down and/or side ways until eventually finding the apple. The snakes behavior during evaluation is indicative that the snake learned some survival behaviors as part of its experience through learning from a Q-table; however, it also indicates that the snake did not develop a strategy for efficiently reaching the apples in a short time frame. Since there were a relatively small number of training steps available to provide the snake with sufficient information about state-actions, this likely limited what useful state-action information could be learned.

```
(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= -5.252, #Apples=14, #Stops=15, #GoodSteps=398, #UniqueStatesVisited=71

** Mean: Return= -5.252, #Apples=14.0, #Stops=15.0, #GoodSteps=398.0, #UniqueStatesVisited=71.0

(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= 23.058, #Apples=23, #Stops=6, #GoodSteps=430, #UniqueStatesVisited=72

** Mean: Return= 23.058, #Apples=23.0, #Stops=6.0, #GoodSteps=430.0, #UniqueStatesVisited=72.0
```

b) Q-table trained with epsilon = 0.6, num_steps = 1000 (qtable_2025.csv)

Training output:

Return = -5.034, Apples = 43, Stops = 13, GoodSteps = 832, UniqueStatesVisited = 70

Evaluation output:

Return = 23.443, Apples = 36, Stops = 27, GoodSteps = 874, UniqueStatesVisited = 71

Observed behavior:

The problem with using the table for Q-learning was that the snake would continue to go in the same direction most of the time and hit the walls. Sometimes, it would even curl up inside itself and force the game to reset. Although the snake could find a relatively good amount of apples, the snake's navigation became unpredictable around walls. Therefore, while exploration helped somewhat, there were still several weaknesses to the policy learned from Q-learning concerning how to avoid terminal states.

```
(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= -5.034, #Apples=43, #Stops=13, #GoodSteps=832, #UniqueStatesVisited=70

** Mean: Return= -5.034, #Apples=43.0, #Stops=13.0, #GoodSteps=832.0, #UniqueStatesVisited=70.0

(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= 23.443, #Apples=36, #Stops=27, #GoodSteps=874, #UniqueStatesVisited=71

** Mean: Return= 23.443, #Apples=36.0, #Stops=27.0, #GoodSteps=874.0, #UniqueStatesVisited=71.0
```

c) Q-table trained with epsilon = 0.4, num_steps = 900 (qtable_2025_4.csv)

Training output:

Return = -2.250, Apples = 29, Stops = 27, GoodSteps = 723, UniqueStatesVisited = 87

Evaluation output:

Return = 31.418, Apples = 58, Stops = 7, GoodSteps = 800, UniqueStatesVisited = 88

Observed behavior:

The snake moved more smoothly compared to lower-epsilon runs. It explored side movements and vertical movements more naturally and avoided walls most of the time. However, it still occasionally hit itself, especially when the snake grew longer. Overall, the behavior was more balanced between exploration and exploitation.

```
(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= -2.250, #Apples=29, #Stops=27, #GoodSteps=723, #UniqueStatesVisited=87

** Mean: Return= -2.250, #Apples=29.0, #Stops=27.0, #GoodSteps=723.0, #UniqueStatesVisited=87.0

(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= 31.418, #Apples=58, #Stops=7, #GoodSteps=800, #UniqueStatesVisited=88

** Mean: Return= 31.418, #Apples=58.0, #Stops=7.0, #GoodSteps=800.0, #UniqueStatesVisited=88.0
```

d) Q-table trained with epsilon = 0.9, num_steps = 1000 (qtable_2025_9.csv)

Training output:

Return = -5.568, Apples = 41, Stops = 20, GoodSteps = 781, UniqueStatesVisited = 79

Evaluation output:

Return = 33.019, Apples = 59, Stops = 8, GoodSteps = 886, UniqueStatesVisited = 80

Observed behavior:

This Q-table was the most effective and consistent. The snake would explore and rarely duplicate its mistakes from before; it would try something different after hitting a wall once, and self-collision was rare until it got too big. This is evidence that exploration during training, which provided the agent with a wider range of state-action values, helped produce the highest quality and most robust behavior.

```
(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= -5.568, #Apples=41, #Stops=20, #GoodSteps=781, #UniqueStatesVisited=79

** Mean: Return= -5.568, #Apples=41.0, #Stops=20.0, #GoodSteps=781.0, #UniqueStatesVisited=79.0

(base) boo@Sreejas-MacBook-Pro-4 snakegame % python QLearning.py
* Run 0: Return= 33.019, #Apples=59, #Stops=8, #GoodSteps=886, #UniqueStatesVisited=80

** Mean: Return= 33.019, #Apples=59.0, #Stops=8.0, #GoodSteps=886.0, #UniqueStatesVisited=80.0
```

Training for the snake using different epsilons (epsilon) and step sizes had vastly different results. Training with lower epsilons and/or less steps often produced either unstable or inefficient behavior. Moderate epsilons produced higher survival rates than low epsilons, but were still prone to producing similar mistakes over and over again. The Q-table produced by training with an epsilon of .09 and 1,000 steps produced the best balance of exploration and exploitation and thus created the smoothest movements, the least amount of wall collisions, and the largest amounts of apples collected.

4. Results and comments for Part 2 experiments

For Part 2, I used the best Q-table from Part 1 (qtable_2025_9.csv) and evaluated it using different epsilon values during evaluation.

Key observations from Part 2:

- Epsilon = 0.1: High apple count but very unstable returns due to limited exploration during evaluation.
- Epsilon = 0.3: Best mean return (10.719) with balanced performance.

- Epsilon ≥ 0.5 : Performance degraded as excessive exploration caused inefficient movement and more stops.
- Epsilon = 0.9: Worst performance with very low apple counts and high number of stops.

These results aligned with my expectations. Low epsilon caused the agent to exploit imperfect policies, while high epsilon caused too much randomness. Moderate epsilon during evaluation produced the most consistent results.

```
## Evaluating with epsilon = 0.1
* Run 0: Return= 5.186, #Apples=56, #Stops=10, #GoodSteps=836, #UniqueStatesVisited=106
* Run 1: Return= -81.476, #Apples=39, #Stops=28, #GoodSteps=825, #UniqueStatesVisited=107
* Run 2: Return= 19.383, #Apples=42, #Stops=28, #GoodSteps=820, #UniqueStatesVisited=108
* Run 3: Return= 27.609, #Apples=43, #Stops=15, #GoodSteps=850, #UniqueStatesVisited=101
* Run 4: Return= 22.920, #Apples=54, #Stops=12, #GoodSteps=812, #UniqueStatesVisited=101
* Run 5: Return= 36.509, #Apples=48, #Stops=22, #GoodSteps=852, #UniqueStatesVisited=97
* Run 6: Return= 11.522, #Apples=45, #Stops=19, #GoodSteps=867, #UniqueStatesVisited=101
* Run 7: Return= 10.821, #Apples=53, #Stops=12, #GoodSteps=841, #UniqueStatesVisited=95
* Run 8: Return= -23.244, #Apples=41, #Stops=28, #GoodSteps=819, #UniqueStatesVisited=103
* Run 9: Return= 0.722, #Apples=27, #Stops=43, #GoodSteps=805, #UniqueStatesVisited=105
** Mean: Return= 2.995, #Apples=44.8, #Stops=21.7, #GoodSteps=832.7, #UniqueStatesVisited=102.4

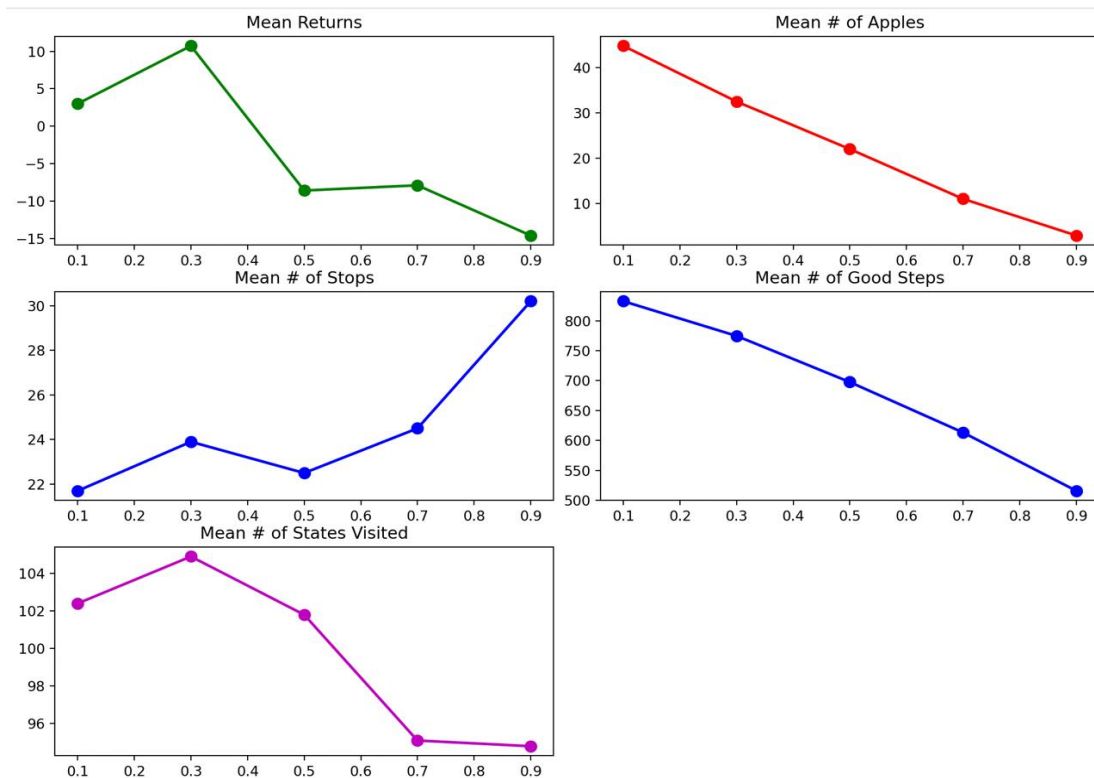
## Evaluating with epsilon = 0.3
* Run 0: Return= 11.702, #Apples=29, #Stops=24, #GoodSteps=793, #UniqueStatesVisited=107
* Run 1: Return= -23.669, #Apples=36, #Stops=21, #GoodSteps=778, #UniqueStatesVisited=109
* Run 2: Return= 9.056, #Apples=26, #Stops=28, #GoodSteps=785, #UniqueStatesVisited=101
* Run 3: Return= 24.976, #Apples=40, #Stops=20, #GoodSteps=755, #UniqueStatesVisited=109
* Run 4: Return= 13.977, #Apples=34, #Stops=19, #GoodSteps=769, #UniqueStatesVisited=106
* Run 5: Return= 4.731, #Apples=31, #Stops=24, #GoodSteps=778, #UniqueStatesVisited=104
* Run 6: Return= 10.417, #Apples=41, #Stops=25, #GoodSteps=752, #UniqueStatesVisited=105
* Run 7: Return= 15.315, #Apples=37, #Stops=16, #GoodSteps=795, #UniqueStatesVisited=94
* Run 8: Return= 16.499, #Apples=27, #Stops=32, #GoodSteps=772, #UniqueStatesVisited=113
* Run 9: Return= 24.184, #Apples=24, #Stops=30, #GoodSteps=770, #UniqueStatesVisited=101
** Mean: Return= 10.719, #Apples=32.5, #Stops=23.9, #GoodSteps=774.7, #UniqueStatesVisited=104.9

## Evaluating with epsilon = 0.5
* Run 0: Return= -75.901, #Apples=25, #Stops=28, #GoodSteps=699, #UniqueStatesVisited=106
* Run 1: Return= 1.736, #Apples=28, #Stops=15, #GoodSteps=708, #UniqueStatesVisited=104
* Run 2: Return= -0.828, #Apples=22, #Stops=17, #GoodSteps=696, #UniqueStatesVisited=92
* Run 3: Return= 10.498, #Apples=26, #Stops=14, #GoodSteps=692, #UniqueStatesVisited=97
* Run 4: Return= 11.374, #Apples=20, #Stops=26, #GoodSteps=704, #UniqueStatesVisited=102
* Run 5: Return= -6.232, #Apples=19, #Stops=25, #GoodSteps=672, #UniqueStatesVisited=115
* Run 6: Return= -3.175, #Apples=23, #Stops=28, #GoodSteps=709, #UniqueStatesVisited=101
* Run 7: Return= 18.329, #Apples=21, #Stops=22, #GoodSteps=693, #UniqueStatesVisited=97
* Run 8: Return= 16.164, #Apples=17, #Stops=22, #GoodSteps=712, #UniqueStatesVisited=103
* Run 9: Return= -57.843, #Apples=19, #Stops=28, #GoodSteps=690, #UniqueStatesVisited=101
** Mean: Return= -8.588, #Apples=22.0, #Stops=22.5, #GoodSteps=697.5, #UniqueStatesVisited=101.8

## Evaluating with epsilon = 0.7
* Run 0: Return= -21.978, #Apples=7, #Stops=27, #GoodSteps=619, #UniqueStatesVisited=94
* Run 1: Return= -27.094, #Apples=16, #Stops=24, #GoodSteps=614, #UniqueStatesVisited=92
* Run 2: Return= 19.811, #Apples=10, #Stops=39, #GoodSteps=594, #UniqueStatesVisited=107
* Run 3: Return= 2.471, #Apples=7, #Stops=38, #GoodSteps=666, #UniqueStatesVisited=101
* Run 4: Return= -8.518, #Apples=13, #Stops=17, #GoodSteps=642, #UniqueStatesVisited=92
* Run 5: Return= -11.264, #Apples=11, #Stops=22, #GoodSteps=591, #UniqueStatesVisited=93
* Run 6: Return= -17.688, #Apples=12, #Stops=22, #GoodSteps=583, #UniqueStatesVisited=97
* Run 7: Return= -4.100, #Apples=10, #Stops=11, #GoodSteps=594, #UniqueStatesVisited=90
* Run 8: Return= 0.282, #Apples=12, #Stops=24, #GoodSteps=604, #UniqueStatesVisited=92
* Run 9: Return= -10.845, #Apples=12, #Stops=21, #GoodSteps=625, #UniqueStatesVisited=93
** Mean: Return= -7.892, #Apples=11.0, #Stops=24.5, #GoodSteps=613.2, #UniqueStatesVisited=95.1

## Evaluating with epsilon = 0.9
* Run 0: Return= -27.212, #Apples=6, #Stops=38, #GoodSteps=542, #UniqueStatesVisited=98
* Run 1: Return= -1.338, #Apples=2, #Stops=35, #GoodSteps=524, #UniqueStatesVisited=102
* Run 2: Return= -81.327, #Apples=3, #Stops=29, #GoodSteps=526, #UniqueStatesVisited=102
* Run 3: Return= 2.598, #Apples=1, #Stops=17, #GoodSteps=528, #UniqueStatesVisited=85
* Run 4: Return= 0.191, #Apples=5, #Stops=23, #GoodSteps=510, #UniqueStatesVisited=93
* Run 5: Return= -22.592, #Apples=3, #Stops=42, #GoodSteps=497, #UniqueStatesVisited=102
* Run 6: Return= 3.572, #Apples=2, #Stops=21, #GoodSteps=497, #UniqueStatesVisited=82
* Run 7: Return= -15.726, #Apples=2, #Stops=34, #GoodSteps=483, #UniqueStatesVisited=100
* Run 8: Return= -1.258, #Apples=2, #Stops=27, #GoodSteps=521, #UniqueStatesVisited=89
* Run 9: Return= -2.879, #Apples=3, #Stops=36, #GoodSteps=529, #UniqueStatesVisited=95
** Mean: Return= -14.597, #Apples=2.9, #Stops=30.2, #GoodSteps=515.7, #UniqueStatesVisited=94.8
```

Graphs:



The figure above illustrate that performance decreases as epsilon grows. At low epsilon values (e.g., 0.1 – 0.3), the snake has more opportunities to collect the apples, lives for longer periods, and receives more rewards (due to more exploitation of the learned Q-table) than at high values of epsilon (e.g., 0.5). Once epsilon is greater than 0.5, the snake acts randomly (less exploitation) and collects fewer apples, makes more stops and does not survive as long. What I find interesting is that the initial number of states visited by the snake first grows as it explores and then falls off at higher values of epsilon, likely because the snake has died too quickly to explore significantly. In general, this project confirmed my suspicion that an overabundance of exploration in evaluation can reduce the performance of a policy that was trained to perform well.

General Reflections

- I have developed a better understanding of how exploration impacts both the quality of learning and the eventual performance of a reinforcement learning model.
- I found this HW to be medium to difficult in terms of finding and eliminating behavior differences when debugging, and confirming that I had properly implemented the logic for evaluating the agent's performance.
- One of the most difficult parts of this was interpreting the noisy results from the experiments and relating the visual observation of the snake's behavior to the numerical metrics.
- I would fine-tune the learning parameters (like alpha, gamma, and decay rate) through grid search or another optimization method instead of manually picking them
- Viewing the snake's behavior visually greatly helped me to understand the numerical results from the project and made the overall learning process much more intuitive.